

Report tesi

Secondo report di avanzamento tesi - aggiornamento al 27 agosto 2026

1. Obiettivo e perimetro del lavoro

Questo report ricostruisce il lavoro svolto dopo il report di agosto 2026. Il punto di arrivo della fase precedente era un insieme di artefatti più robusto, un training set supervisionato, un primo dataset per LLM con vincoli espliciti di provenienza e validazione e lo scheletro del prototipo che utilizzerà l'LLM. A questo punto ci siamo resi conto che ci stavamo allontanando dall'obiettivo primario di questa tesi, cioè quello di avere un modello che, dato un circuito e una figure of merit in input, ci consigli l'hardware adatto e la relativa configurazione Qiskit.

Il lavoro svolto da qui in avanti è quindi derivato dalla comprensione di questo allontanamento.

Infatti l'obiettivo è stato costruire un dataset adatto a questo obiettivo.

Il dataset deve quindi contenere esempi storici con ognuno queste componenti:

il circuito logico, la figure of merit, il device migliore, la configurazione migliore, claim + evidence.

Oltre alla ristrutturazione del dataset, sono stati strutturati e implementati i seguenti punti:

- Gestione dei vincoli dell'utente
- Definizione formale di circuiti "simili" (necessaria per un retrieval efficace)
- Politica di retry LLM

2. Stato iniziale: da dove sono partito

2.1 Dataset adatto

Partendo dalla situazione descritta in precedenza, ho ritenuto saggio (grazie anche ai consigli) di scartare tutta la parte che riguardava le trace RL, che per quanto interessanti, non aiutavano realmente a risolvere il problema posto.

Partendo da qui ho quindi creato un nuovo branch di lavoro in cui ho rimosso tutto ciò che riguardava le trace RL e i modelli ML.

In questo nuovo branch "qiskit_dataset" ho voluto tenere solo ciò che fosse strettamente necessario per rispondere alla domanda principale di questa tesi.

Il branch main con tutto il lavoro precedente rimane invariato e solido su github, riutilizzabile nel caso di futuri miglioramenti.

Come detto prima, il dataset doveva contenere esempi etichettati che descrivessero come una risposta dell'LLM debba essere: quindi composta da circuito logico, la figure of merit, il device migliore, la configurazione migliore, claim + evidence.

L'obiettivo quindi è quello di, a partire da circuiti di training selezionati, costruire un dataset con la struttura descritta.

Questo porta inevitabilmente a dover definire formalmente che cosa è una configurazione qiskit.

I parametri di qiskit.transpile sono tanti, ma per questa prima fase del lavoro, mi sono concentrato su quelli che ritenevo più importanti alla compilazione che sono: hardware, layout_method,

routing_method e optimization_level.

Questi 4 parametri sono essenzialmente i responsabili delle fasi più fragili e importanti della compilazione; perciò, ho ritenuto una buona scelta partire da questi.

2.2 Struttura e validazione vincoli utente

I vincoli dell'utente non erano ancora stati definiti.

Una possibilità era quella di far scrivere in linguaggio naturale i vincoli all'utente, ma mi sono reso conto presto che sarebbe stato un inutile aumento di complessità per un task di minor importanza.

Inoltre è importante definire quali vincoli l'utente può esprimere, poiché non tutti i vincoli possono essere imposti. Ad esempio è difficile rispettare un vincolo che imponga una certa struttura al circuito compilato, o ancora è difficile sapere a priori le tempistiche della compilazione.

I vincoli più semplici da descrivere e rispettare sono invece quelli legati all'hardware, in particolare alle loro caratteristiche.

Ho quindi deciso di dare all'utente la possibilità di esprimere vincoli solo sugli hardware, e applicare quindi a tutti gli effetti dei "filtri" sul catalogo di hardware.

Questi vincoli e la loro struttura formale verranno discussi nella sezione 3.2

2.3 Similarità tra circuiti

Un'altra formalizzazione importante è quella della "similarità" tra circuiti.

Infatti, in fase di retrieval dell'informazione dal dataset, la domanda che mi sono posto è stata quella di "come decidere quali esempi selezionare come aumento del prompt per l'LLM?".

Infatti non basta avere un grande dataset con tanti esempi per risolvere il problema. Dato che la finestra di contesto dell'LLM ha un limite, è necessario selezionare in modo attento solo gli esempi più simili e pertinenti che possono aiutare al meglio l'LLM nel scegliere l'hardware e la configurazione migliore per la richiesta dell'utente, e per fare ciò bisogna avere una definizione formale di "qual è l'esempio migliore?"

L'intuizione mi ha portato a pensare che se il circuito di input ha struttura e proprietà simili a qualche altro circuito di esempio, allora la sua compilazione potrebbe essere abbastanza simile.

Da questa intuizione ho cercato come poter accertarsi scientificamente che due circuiti siano simili.

Una prima e semplice soluzione è quella di calcolare la distanza tra i feature vectors dei due circuiti, basta infatti vedere quale coppia di circuiti ha la distanza minore e quello potrebbe essere il miglior candidato. Ma questo metodo parte con l'assunzione che la somiglianza strutturale di due circuiti implica anche la somiglianza comportamentale. Per ora potrebbe rimanere comunque un primo metodo valido per avere un qualche tipo di valutazione su quale circuito sia più simile.

Un'opzione più complessa da prendere in considerazione in futuro è il Quasi-Isomorfismo tra DAG.

Un circuito quantistico può essere rappresentato come un grafo, in particolare come un DAG, dove i nodi sono le porte e gli archi sono i qubit/clbit.

Quindi due DAG identici rappresentano lo stesso circuito, si dicono cioè isomorfi.

Esistono algoritmi di controllo dell'isomorfismo polinomiali, ma non è sufficiente per questa task, poiché la maggior parte dei circuiti ha un DAG diverso, e inoltre il nostro obiettivo è trovare circuiti simili nel comportamento, non identici, per questo ho cercato se fosse possibile vedere se due DAG fossero

“quasi-isomorfi”.

Un quasi-isomorfismo è un indebolimento dell’isomorfismo: significa che due oggetti non sono identici in tutto e per tutto, ma lo diventano quando vengono guardati attraverso la lente di una specifica proprietà fondamentale (solitamente l’omologia o la coomologia).

In particolar modo questi 2 algoritmi calcolano questa somiglianza:

- Weisfeiler-Lehman (WL) Graph Kernel: Confronta la struttura dei grafi contando sotto-alberi simili. (da preferire per circuiti con tanti qubit)
- Graph Edit Distance (GED): Calcola il numero minimo di operazioni (aggiunta/rimozione nodi o archi) per trasformare il Circuito A nel Circuito B. (da usare per circuiti con pochi qubit poiché NP-Hard)

Per ora mi attengo alla semplice distanza vettoriale tra i feature vectors dei circuiti.

2.4 Politica di retry LLM

Per poter definire la politica di retry dell’LLM in caso di output non valido, bisogna prima formalizzare il concetto di output non valido.

La risposta dell’LLM è composta da: hardware consigliato + configurazione qiskit consigliata + claim ed evidence.

Per poter dire quindi che la risposta è valida, bisogna vedere se tutte le 3 componenti sono valide.

- Hardware: per controllare che sia valido basta controllare che sia contenuto nel catalogo degli hardware e che rispetti i vincoli dell’utente, il che è un controllo molto semplice e veloce.
- configurazione qiskit: come per l’hardware, basta vedere che la configurazione appartenga all’insieme delle configurazioni ritenute valide e che rispetti i vincoli dell’utente.
- claim ed evidence: qui invece è più difficile verificare la validità di una motivazione in linguaggio naturale.

Per fare questo infatti dovremmo avere qualche modo deterministico per capire se la motivazione data dall’LLM è effettivamente sensata, o se non è coerente.

Poiché claim + evidence è stato strutturato in modo tale da riferirsi sempre alle informazioni dei circuiti storici del dataset, un modo per capire se la motivazione è coerente, è vedere che abbia fatto riferimenti espliciti a circuiti di esempio da cui ha ottenuto l’informazione tramite retrieval.

Infatti una prima differenza da notare è che i claim + evidence presenti nel dataset, sono ovviamente dei claim oggettivi, poiché, avendo compilato prima tutte le combinazioni possibili, si sa già qual è la migliore opzione e quindi è facile dare come claim un semplice “questo hardware è il migliore perché è quello con lo score più alto”.

Quando invece l’LLM riceverà nuovi circuiti, il suo claim + evidence dovrà sembrare qualcosa come:

“Raccomando quantinuum_h2_56. Tra gli otto casi storici maggiormente simili, questo device ha ottenuto il risultato migliore in sei casi e presenta il miglior ranking medio pesato. Il nuovo circuito condivide con tali casi il numero di qubit, una percentuale simile di operazioni a due qubit e una struttura analoga del grafo d’interazione. Sul device raccomandato, la configurazione indicata è stata la più efficace nel 68% pesato dei casi recuperati. Il circuito corrente non è ancora stato compilato, quindi la raccomandazione rappresenta una previsione basata su casi analoghi.”

Questa risposta verrà generata dall’LLM usando come supporto i dati oggettivi racchiusi nel dataset, non verrà chiesto al modello di “tirare a indovinare” le statistiche solo per analogia/somiglianza tra circuiti

3. Che cosa ho deciso di fare

3.1 Come strutturare il dataset

Per strutturare il dataset era necessario capire quante combinazioni possibili ci fossero.

Ogni singola run è composta da: circuito x figure of merit x hardware x configurazione qiskit x seed

Per una prima fase del dataset ho congelato alcune opzioni tra cui:

- figure of merit: per ora solo expected_fidelity, in futuro potremmo aggiungere anche critical_depth
- hardware: solo 5, ibm_falcon_27, ibm_falcon_127, ibm_heron_133, ibm_heron_156, quantinuum_h2_56, poiché sono gli stessi di cui abbiamo dei modelli RL pronti per il futuro confronto con MQT-Predictor
- configurazioni qiskit: come descritto nel punto 2.1, layout_method, routing_method e optimization_level sono i 3 parametri principali che fanno parte della configurazione, e ognuno di loro ha da 4 ai 5 possibili valori

layout_method: default, dense, sabre, trivial → 4 valori

routing_method: default, basic, lookahead, sabre, none → 5 valori

optimization_level: 0,1,2,3 → 4 valori

Le combinazioni totali, escludendo i circuiti, risultavano: 5 (hardware) x (4x4x5) (configurazioni) x 3 (seed diversi) = 1200 configurazioni per singolo circuito. Con 600 circuiti sarebbero 720000 compilazioni.

Per snellire le possibili combinazioni, ho deciso di escludere optimization_level 0 e 1, poiché non adatti quando si cerca un circuito di qualità, piuttosto che la velocità, e ho escluso anche routing_method none, perché se un circuito necessita di un routing method, con none fallisce, e quindi occuperebbe tempo inutilmente.

Quindi dalle 1200 configurazioni per circuito siamo passati a 480 configurazioni, che per 600 circuiti diventano 288000 compilazioni.

La struttura completa con spiegazione di ogni schema e file è presente in datasets/expected_fidelity/README.md

3.2 Vincoli utente

In questa fase iniziale ho pensato di poter dare all'utente la possibilità di selezionare vincoli controllati e non in linguaggio naturale.

Ho ritenuto saggio dare all'utente la possibilità di scegliere alcuni vincoli riguardanti gli hardware, tra cui:

-Provider ammesso (es. Solo IBM, IBM e Quantinuum, solo Rigetti ecc..)

-Hardware ammessi (es. solo ibm_falcon_127 e ibm_heron_133)

- Min e Max numero di qubit dell'hardware (es. min = 20, max = 80)
- Gate nativi obbligatoriamente presenti (es. cx, id, rz)

L'idea è quella di poter dare all'utente l'opzione di "filtrare" i dispositivi in base a questi 4 vincoli scelti da me, in questo modo è perfettamente validabile e facilmente implementabile.

Inoltre in questo modo è possibile creare un oggetto JSON che rappresenta la richiesta dell'utente, che sarà composta da circuito in input, figure of merit scelta e vincoli selezionati.

Il prototipo è uno scheletro modulare, ogni modulo ha il suo compito e i moduli devono comunicare tra di loro in modo semplice e veloce. Questo per dire che una richiesta strutturata come un oggetto JSON è molto semplice da scambiare tra un modulo e l'altro.

Dettagli maggiori sulla struttura effettiva del JSON si trovano in `prototype/README.md`

3.3 Similarità tra circuiti

Per ora la similarità tra circuiti rimane implementata come distanza tra i vettori delle feature dei circuiti. Non è dimostrato né verificato che due circuiti si comportino similmente se le loro feature sono vicine, infatti i loro DAG potrebbero essere comunque diversi, le operazioni di gate potrebbero comunque modificare sensibilmente i loro comportamenti ecc...

3.4 Validazione dell'output e politica di retry LLM

Come discusso nella sezione 2.4, la difficoltà nel validare l'output LLM sta quasi esclusivamente nel validare la claim + evidence. Infatti, se device e configurazione rientrano tra quelli validi, allora l'output rimane valido, indipendentemente dalle performance di compilazione.

Mentre hardware e configurazione sono semplici da validare, poiché basta riapplicare i controlli che vengono eseguiti per la validazione della richiesta utente e un controllo generale sull'appartenenza al catalogo definito, claim ed evidence invece sono la parte meno deterministica.

Un modo di controllare la claim è rendere l'LLM capace di argomentare delle statistiche oggettive, cioè il suo claim deve descrivere che la scelta che ha fatto è dovuta da un insieme di statistiche sugli esempi storici più simili all'input, e quelle statistiche sono oggettive e non confutabili.

Quindi nel concreto, all'LLM vanno passati tramite retrieval le informazioni e statistiche oggettive dei casi passati più simili al caso attuale, in questo modo, quando farà la sua scelta, potrà spiegarla usando proprio quelle statistiche.

Attualmente non è ancora implementata la fase di retry e validazione dell'output.

4. Quali test ho eseguito

Tutti i test sono racchiusi negli script di test presenti in `tests/`

Per ora le uniche parti implementate di quelle descritte fino ad ora sono la gestione e validazione dei vincoli utente, che è testata dallo script `tests/test_request_constraints.py`; e la generazione del dataset qiskit secondo le configurazioni scelte e descritte nella sezione 3.1, gli script sono:

`/home/elio/Tesi/tests/test_qiskit_dataset.py` e `/home/elio/Tesi/tests/test_qiskit_dataset_aggregation.py`

Test più concreti verranno disposti quando il dataset verrà completato per tutti i 600 circuiti.

5. Quali risultati ho ottenuto

5.1 Datasets

Ho popolato il dataset con 10 circuiti esempio per testare la generazione e la struttura di quest'ultimo, il tutto è dentro `datasets/expected_fidelity/pilot`

Il tutto usando gli script dentro `qiskit_dataset` che vengono usati negli script 07,08,09 e 10 dentro `scripts/`

5.2 Prototipo LLM

Il prototipo è stato esteso con l'implementazione della costruzione e validazione della richiesta utente

6.Domande e prossimi steps

Riferimenti

- [1] N. Quetschlich, L. Burgholzer, R. Wille, MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing, ACM Transactions on Quantum Computing, 2025.
- [2] N. Quetschlich, L. Burgholzer, R. Wille, Predicting Good Quantum Circuit Compilation Options, 2023.
- [3] Munich Quantum Toolkit, MQT Predictor 2.3.0: documentazione e codice sorgente analizzati nell'ambiente di lavoro.